

Programação de Aplicações Web

Aula 7: Node.js, Frameworks Frontend e Introdução ao React



O que é Node.js?

JavaScript além do navegador.

Node.js: O Motor por Trás do Backend Moderno

Node.js é um ambiente de execução JavaScript **open-source**, **cross-platform**, que permite executar código JavaScript fora de um navegador web.

- **V8 Engine:** Utiliza o motor de alto desempenho do Google Chrome.
- **Event-Driven:** Baseado em eventos e I/O não bloqueante.
- **Ecosistema:** NPM (Node Package Manager) é o maior registro de software do mundo.

Antes do Node.js

JS era usado apenas para interatividade no frontend (validação de formulários, animações).

Depois do Node.js

JS se tornou uma linguagem "full-stack", capaz de gerenciar servidores, bancos de dados e APIs.

Como o Node.js funciona? (Arquitetura)

O segredo está no **Event Loop** e no **I/O Não Bloqueante**.

- **Single Threaded:** Executa em uma única trilha principal.
- **Non-blocking:** Não espera a conclusão de uma tarefa pesada (como leitura de arquivo) para continuar.
- **Asíncrono:** Utiliza *callbacks*, *promises* e *async/await*.



Event Loop

Gerencia todas as requisições, delegando tarefas pesadas para o sistema operacional e retomando quando prontas.

"O Node.js é como um garçom eficiente que anota os pedidos e os entrega à cozinha, mas não fica parado esperando o prato ficar pronto para atender o próximo cliente."

Por que usar Node.js no Desenvolvimento Web?

As vantagens de unificar o ecossistema.

- **Uma Linguagem Única:** JavaScript no Frontend e no Backend (Full Stack JS).
- **Velocidade de Desenvolvimento:** Bibliotecas prontas para quase tudo no NPM.
- **Escalabilidade:** Excelente para aplicações de tempo real (Chats, Streaming, Colaboração).
- **Grande Comunidade:** Suporte massivo e documentação abundante.



PERFORMANCE



PRODUTIVIDADE



UNIVERSALIDADE

A Evolução do Desenvolvimento Frontend

De páginas estáticas a aplicações complexas.

Do jQuery aos Frameworks Modernos

O desenvolvimento web mudou drasticamente nos últimos 15 anos.

Era	Tecnologia Principal	Característica
Estática	HTML / CSS Puro	Páginas fixas, pouco dinamismo.
DOM Manipulation	jQuery	Manipulação manual e direta do DOM.
MVC Frontend	AngularJS / Backbone	Tentativa de organizar código em grandes apps.
Componentes	React / Vue / Svelte	UIs baseadas em peças reutilizáveis e estados.

A complexidade das interfaces modernas exigiu ferramentas mais robustas.

Principais Frameworks e Bibliotecas Atuais

O "Big Three" (e um bônus).

React

Biblioteca focada em componentes. Criada pelo Facebook (Meta). A mais popular do mercado.

Angular

Framework completo (batteries-included) do Google. Forte em grandes empresas.

Vue.js

Progressivo e fácil de aprender. Criado por Evan You. Equilíbrio entre React e Angular.

Svelte

"Compila" o código fora do navegador. Sem Virtual DOM, extremamente rápido.

Nesta aula, focaremos no React.

Introdução ao React

A biblioteca para interfaces de usuário.

O que torna o React Especial?

React não é um framework, é uma **biblioteca**. Ele foca exclusivamente na camada de visualização (View).

- **Declarativo:** Você descreve o *que* quer na tela, e o React cuida de *como* atualizar.
- **Baseado em Componentes:** Construa peças pequenas que, juntas, formam uma interface complexa.
- **Virtual DOM:** Uma cópia leve do DOM real que permite atualizações ultra-rápidas.
- **Aprenda uma vez, escreva em qualquer lugar:** React Native permite criar apps móveis com o mesmo conhecimento.



O Poder da Componentização

Pensar em interfaces como um conjunto de peças.

Imagine um site como um conjunto de peças de **LEGO**.



Independência

Cada componente tem sua própria lógica e estilo.



Reutilização

Use o mesmo botão ou card em várias partes do site.



Manutenção

Mude um componente em um lugar e todos se atualizam.

```
<Header />  
<Sidebar />  
<MainContent>  
  <UserList />
```

Componentes Funcionais

A forma moderna de escrever componentes (usando Funções JS).

```
function Welcome(props) {  
  return (  
    <div className="card">  
      <h1>Olá, {props.name}</h1>  
      <p>Bem-vindo ao mundo do React!</p>  
    </div>  
  );  
}
```

// Para usar:
// <Welcome name="Paulo" />

- **Simples:** Apenas uma função que retorna HTML (JSX).
- **Hooks:** Permitem adicionar estado e outras funcionalidades.
- **Props:** Forma de passar dados para dentro do componente (como argumentos de função).

JSX: JavaScript XML

O "açúcar sintático" que amamos (ou estranhamos no começo).

JSX permite escrever código que se parece com HTML dentro do JavaScript.

```
const element = <h1>Olá, Mundo!</h1>;

function MyButton() {
  const label = "Clique aqui";
  return <button className="btn">{label}</button>;
}
```

Por que usar?

- Mais legível.
- Erros de compilação ajudam no debug.
- Lógica e UI ficam no mesmo arquivo.

Importante:

- Atributos usam camelCase (*className* em vez de *class*).
- Variáveis JS entram entre chaves `{ }`.

Single Page Applications (SPA)

O conceito que revolucionou a experiência do usuário.

Diferente do modelo tradicional (MPA), onde cada clique carrega uma nova página HTML do servidor:

Multipage (Tradicional)

- ❌ Tela pisca a cada clique.
- ❌ Servidor gera o HTML todo vez.
- ❌ Navegação mais lenta.

Single Page (SPA)

- ✅ Carrega o HTML apenas uma vez.
- ✅ Atualiza apenas o conteúdo necessário.
- ✅ Sensação de App nativo (fluidez).

O React é a ferramenta perfeita para gerenciar essas trocas de conteúdo dinâmicas.

Como o React é tão rápido? Virtual DOM

Manipular o DOM real do navegador é custoso e lento.

1. **Cópia em Memória:** O React mantém uma representação leve do DOM (o Virtual DOM).
2. **Diffing:** Quando algo muda, o React compara o novo Virtual DOM com a versão anterior.
3. **Reconciliation:** Ele identifica exatamente o que mudou e atualiza **apenas** essas partes no DOM real.



React Hooks: useState

Gerenciando dados dinâmicos (Estado).

O **Estado** é a memória do componente. Quando o estado muda, o React atualiza o componente automaticamente.

```
import { useState } from 'react';

function Contador() {
  const [contagem, setContagem] = useState(0);

  return (
    <div>
      <p>Você clicou {contagem} vezes</p>
      <button onClick={() => setContagem(contagem + 1)}>
        Aumentar
      </button>
    </div>
  );
}
```

Sintaxe: `const [valor, setValor] = useState(valorInicial);`

React Hooks: useEffect

Lidando com "Efeitos Colaterais".

Use o `useEffect` para ações que acontecem fora do fluxo de renderização (chamadas de API, timers, etc).

```
import { useEffect } from 'react';

function MyComponent() {
  useEffect(() => {
    console.log("O componente foi montado!");
  }, []); // Array de dependências vazio = executa apenas uma vez

  return <div>Olhe o console!</div>;
}
```

Sem array: Executa em toda renderização.

Com variáveis no array: Executa quando elas mudarem.

Comunicação entre Componentes (Props)

Passando dados de pai para filho.

No React, os dados fluem em uma única direção (Top-Down).

```
// Componente Filho
function UserCard({ name, role }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>{role}</p>
    </div>
  );
}

// Componente Pai
function App() {
  return (
    <UserCard name="Paulo" role="Desenvolvedor" />
  );
}
```

"Props são imutáveis. Um componente nunca deve modificar suas próprias props."

Navegação com React Router

Mantendo a experiência SPA com múltiplas "páginas".

O `react-router-dom` permite que você altere a URL e o conteúdo sem recarregar o navegador.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> | <Link to="/about">Sobre</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Importante: Use `<Link>` em vez de `<a>` para evitar recarregamento.

Requisições HTTP com Axios

Buscando dados de APIs externas.

O **Axios** é uma biblioteca popular para fazer requisições HTTP, oferecendo uma API mais limpa que o `fetch` nativo.

```
import axios from 'axios';
import { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    axios.get('https://api.github.com/users')
      .then(response => setUsers(response.data))
      .catch(error => console.error(error));
  }, []);

  return (
    <ul>
      {users.map(user => <li key={user.id}>{user.login}</li>)}
    </ul>
  );
}
```

Projeto Prático: DevFinder

Vamos ver tudo isso funcionando junto?

Confira a pasta `exemplos/github-explorer` para uma aplicação completa que utiliza:

- **Vite** como build tool.
- **Axios** para buscar usuários do GitHub.
- **useState** para gerenciar a busca e os resultados.
- **React Router** para navegação entre Início e Sobre.
- **Props** para passar dados ao card de usuário.

Conclusão e Revisão

Consolidando o conhecimento.

O que aprendemos hoje?

Backend: Node.js e sua arquitetura assíncrona.

Ecossistema: A evolução dos frameworks modernos.

React Base: Componentes, JSX e Props.

React Avançado: Hooks (useState, useEffect).

Navegação: React Router para SPAs.

Dados: Consumo de APIs com Axios.

Próximo passo: Projeto Prático!